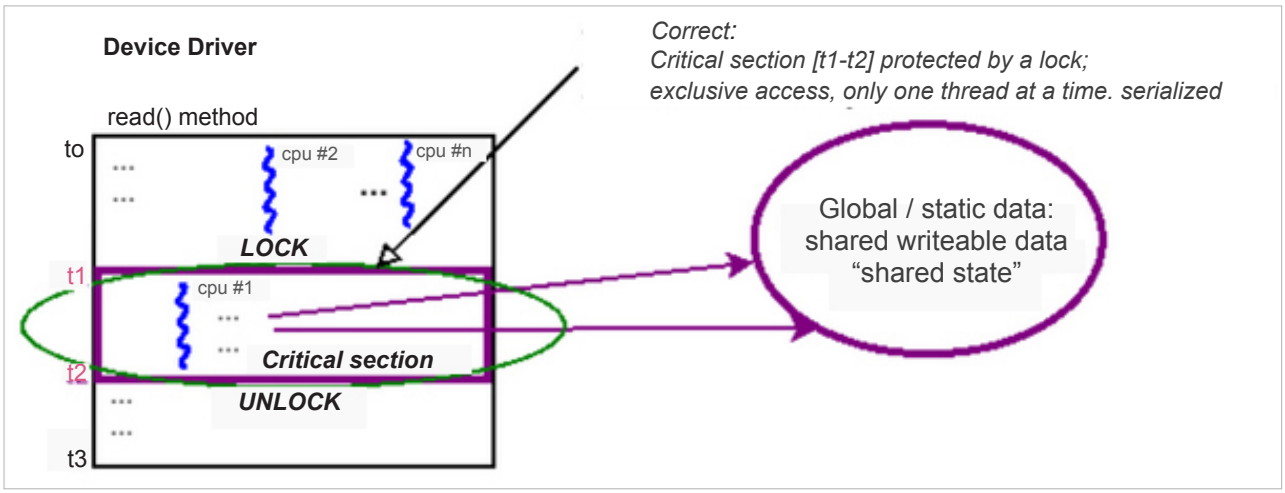

Figure 12.3: A conceptual diagram showing how a critical section code path is honoured, given exclusivity, by using a lock

(the critical section is effectively the code between the *lock* and the *unlock* operations!). What happens to the n-1 "loser" threads? They (perhaps) see the lock API as a blocking call; they, to all practical effect, wait. Wait upon what? The *unlock* operation, of course, which is performed by the owner of the lock (the "winner" thread)! Once unlocked, the remaining n-1 threads now compete for the next "winner" slot; of course, exactly one of them will "win" and proceed forward; in the interim, the n-2 losers will now wait upon the (new) winner's *unlock;* this repeats until all n threads (finally and sequentially) acquire the lock.

Now, locking works of course, but – and this should be quite intuitive – it results in (pretty steep!) **overhead, as it defeats parallelism and serializes** the execution flow! To help you visualize this situation, think of a funnel, with the narrow stem being the critical section where only one thread can fit at a time. All other threads get choked; locking creates bottlenecks.

Another oft-mentioned physical analog is a highway with several lanes merging into one very busy – and choked with traffic – lane (a poorly designed toll booth, perhaps). Again, parallelism – cars (threads) driving in parallel with other cars in different lanes (CPUs) – is lost, and serialized behavior is required – cars are forced to queue one behind the other.

Thus, it is imperative that we, as software architects, try and design our products/projects so that locking is minimally required. While completely eliminating global variables is not practically possible in most real-world projects, optimizing and minimizing their usage is required. We shall cover more regarding this, including some very interesting lockless programming techniques, later. Another really key point is that a newbie programmer might naively assume that performing reads on a shared writeable data object is perfectly safe and thus requires no explicit protection (with the exception of an aligned primitive data type that is within the size of the processor's bus); this is untrue. This situation can lead to what's called **dirty or torn reads,** a situation where possibly stale data can be read as another writer thread is simultaneously writing while you are – incorrectly, without locking – reading the very same data item.

Since we're on the topic of atomicity, as we just learned, on a typical modern microprocessor, the only things guaranteed to be atomic are a single machine language instruction or a read/write to an aligned primitive data type within the processor bus's width. So, how can we mark a few lines of "C" code so that they're truly atomic? In user space, this isn't even possible (we can come close, but cannot guarantee atomicity).

How do you "come close" to atomicity in user space apps? You can always construct a user thread to employ a SCHED_FIFO policy and a real-time priority of 99. This way, when it wants to run, pretty much nothing besides hardware interrupts/exceptions can preempt it. (The old audio subsystem implementation heavily relied on this.)

In kernel space, we can write code that's truly atomic. How, exactly? The short answer is that we can use spinlocks! We'll learn about spinlocks in more detail shortly.

A summary of key points

Let's summarize some key points regarding critical sections. It's really important to go over these carefully, keep these handy, and ensure you use them in practice.

- A **critical section** is a code path that can execute in parallel and that works upon (reads and/or writes) shared writeable data (also known as "shared state").
- Because it works on shared writable data, the critical section requires protection from the following:
  - Parallelism (that is, it must run alone/serialized/in a mutually exclusive fashion)